Conversational AI Platform in Banking Sector

Riya Kulkarni
Department of Computer Engineering
MKSSSs Cummins College of Engineering for Women
Pune, India
riya.kulkarni@cumminscollege.in

Akshata Jedhe
Department of Computer Engineering
MKSSSs Cummins College of Engineering for Women
Pune, India
akshata.jedhe@cumminscollege.in

Payal Mehta
Department of Computer Engineering
MKSSSs Cummins College of Engineering for Women
Pune, India
payal.mehta@cumminscollege.in

Juee Ashtaputre
Department of Computer Engineering
MKSSSs Cummins College of Engineering for Women
Pune, India
juee.ashtaputre@cumminscollege.in

***Abstract*—A chatbot that encourages human discussion through text or speech is known as a conversational AI platform. It allows users to communicate with digital gadgets as if they were talking to actual people. Chatbots may imitate human communication using AI-powered NLP. They may deduce the text's underlying intent as the real person is typing, and then offer a response that corresponds to the intent.**

Keywords— chatbot, natural language processing, RASA, banking chatbot, RASA NLU, RASA core

I. Introduction

Technological advancements in the current market require industries to change and update their procedures. The banking industry is one of the most advanced, and it is constantly seeking for new technical solutions to increase its efficiency.Netbanking websites are complicated, requiring you to navigate through several pages to obtain the information you want. When dealing with consumers directly, bank employees face several unpleasant scenarios. Chatbots may be used to gracefully avoid such situations. The banking industry has also realized the game-changing potential of breakthrough technological disruptors such as Artificial Intelligence (AI) and has worked quickly to improve its online and mobile banking models with consumer contact platforms.

According to a Salesforce survey, just 32% of organizations in the financial industry now utilize AI chatbots, with 37% aiming to do so within the next 18 months. This yields a prospective growth rate of 118 percent, indicating strong industry demand.

II. Chatbots and banking field

Customers' needs were only intensified by Covid-19 and the pandemic's repercussions. Consumers want immediacy, personalization, and seamless experiences from their favorite businesses, and they expect the same from their financial institutions. It's no wonder, therefore, that the new generation of digital banking is focused on the client experience.

Banks must increase the quality of their customer service while avoiding wasting time on unnecessary consumer inquiries. As a result, they now see the value of automation and 24-hour services that benefit not only them but also their clients. This entails offering scalable, round-the-clock customer service through numerous channels and languages.They want sophisticated platforms that can communicate with these clients and comprehend their messages. Intelligent chatbots are required. Customers, as well as staff, may benefit from individualized help via their preferred channel and language. Chatbots do not need to be a threat to human agents because their purpose is to increase employee productivity by letting them to focus on more complicated jobs that need their expertise and ability, while lowering support costs and boosting customer retention.

Chatbots are expected to save banks billions of dollars over the next decade. Chatbots will save over $8 billion in yearly costs by 2022, according to a Juniper research. According to Gartner, chatbots will handle at least 85% of all customer care interactions by 2020.

The most prevalent application case for chatbots in banking customer service is the automation of repeated monotonous activities such as banking queries, EMI calculators, and so on. Such activities necessitate quick and straightforward answers that necessitate self-learning but do not necessitate originality.Chatbots might be a solution to all or some of these issues. Chatbots may build a valuable database of future users by automating consumer enquiries.

Users will communicate with the chatbot. Rasa will determine the purpose based on the questions asked and deliver a suitable response from the database. Web scraping was used to construct the database.

III. Building a conversational platform

The Conversational AI platform is built using RASA.

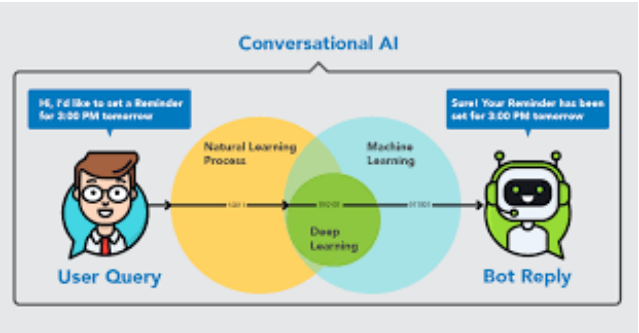
1. ***Environment***: RASA : This is where the core Natural Learning Process (NLP) engine and context interpretation happens.
2. ***Web Framework :*** FLASK
3. ***Database :*** SQL
4. ***Traffic Server:*** Apache Server : The server that handles the traffic requests from users and routes them to appropriate components. The traffic server also routes the response from internal components back to the front-end systems.
5. ***Integration Component:***
   - REST APIs for connecting to database
   - ServiceNow Integration to raise tickets
   - Connect to Live Agent or Call Center

IV. Literature survey

Conversational assistants are becoming an integral part of daily life. Chatbots are a promising technology with the potential to enhance workplaces and everyday life. In terms of scalability and accessibility, they also offer unique possibilities as communication and information tools for digital learning.

Rasa Core and Rasa Natural Language understanding (NLU) are easy to use tools for building conversational systems Rasa is an essential set of tools for building more advanced and efficient AI assistants/chatbots. The benefit of rasa is the infrastructure and tools which provides the user with high performance, resilient and proprietary intelligent chatbots that work. Rasa helps all developers create better text and voice-based chatbots. Rasa's NLU helps the developers with the technology and the tools necessary for capturing and understanding user input, determining the intent and entities. Rasa supports multiple languages, single and multiple intent, and both pre-trained and custom entities

Rasa is an open source framework for building AI bots. It is very convenient and easy to customize. Rasa open source framework consists of two components: - Rasa NLU and Rasa core. Rasa recommends using both Rasa NLU and core, but they can be used independently of each other. Rasa core is the component which handles the dialogue engine for the framework and helps in creating more complex chatbots with customization. Rasa provides an opportunity for interactive learning. Chatbots can be enhanced because of the flexibility options provided by the Rasa framework. The chatbot can be easily deployed, integrated and connected to websites and applications



A. *Overview of RASA*

Rasa Core and Rasa Natural Language Understanding (NLU) are simple conversational system construction tools. Rasa is a suite of technologies that may be used to create more powerful and efficient AI assistants and chatbots. The infrastructure and resources provided by rasa allow users with high-performing, robust, and proprietary intelligent chatbots that operate. All developers may use Rasa to build better text and voice-based chatbots. Rasa's NLU provides developers with the technology and tools they need to capture and analyze user input, as well as determine intent and entities. Rasa can handle several languages, numerous intents, and both pre-trained and bespoke entities.

Rasa is an open source platform for creating artificial intelligence bots. It's incredibly user-friendly and customizable. The Rasa open source framework is divided into two parts: Rasa NLU and Rasa core. Rasa suggests utilizing Rasa NLU and core together, however they may be used separately. Rasa core is the component that manages the framework's dialogue engine and aids in the creation of more complicated chatbots with customisation. Rasa provides an interactive learning environment. Because of the flexibility choices given by the Rasa framework, chatbots may be improved. The chatbot is simple to set up, integrate, and connect to websites and apps.

Components of RASA:

- Stories.yml: Contains the conversational flow of the chatbot and the user, which we have trained
- config.yml: Contains the configuration we set up for our NLU and Core models. Policies, pipelines, etc are defined here.
- endpoints.yml: Contains the different endpoints the chatbot can utilize. Exposes the chatbot to the Custom Action server and the UI.
- actions.py: Contains the main logic of the chatbot. Actions define the responses of the chatbot.
- domain.yml: Defines the boundaries of the chatbot and the guidelines on how the chatbot should function
- NLU.yml: This file contains the training data. It basically consists of intent examples ie user input

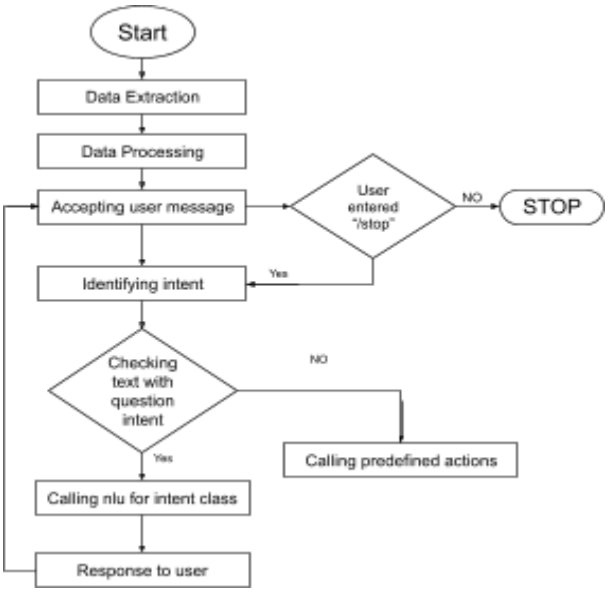
B. *Application logic*

Application Logic consists of dialog management tools , NLU toolkit and other features that support functionalities of the application. Controller interacts with the modules of the Application Logic and the database to fulfill the requests of the user.

C. *Project Design*[1]

- Web scraping : Given a website from which we have to extract the FAQ data and store it in a proper format. For this request library is used to send http requests to the website and the request returns a response object.
- Then we beautifulsoup python library is used for web scraping on the response we received to access the website content and to extract meaningful and required data. Then the data is stored in the form of a csv file. Here we have extracted data into 2 csv files i.e question and answer.
- The system has two main entities as User and Chatbot which interact with each other. As the user will be chatting to the bot, the user message will be classified according to the correct intent. Then the answer associated with that intent will be searched and extracted from the database or if there is some action related to the intent that action is called and the corresponding answer is displayed back to the user.

Flowchart of working of chatbot



D. Project Modules

- ***Web Scraping using beautiful Soup:*** With web scraping, we can access the HTML of the web page and extract useful information/data from it. Beautiful Soup is a Python library that is used for web scraping purposes to pull the data out of HTML and XML files. It creates a parse tree from page source code that can be used to extract data in a hierarchical and more readable manner. We are using web scraping to extract data from the internet to csv files.
- ***Data Preprocessing:*** We have removed duplicate entries and special characters from the extracted data using pandas library.
- ***Rasa Intent Classification:*** Rasa NLU will classify the user messages into one or also multiple user intents. We are using a pre-trained embedding for the classification.
- ***EMI Calculator:*** Created necessary intents which rasa nlu will classify. Then the input values / details necessary for EMI calculation are taken using action forms in Rasa and then a custom action is made to calculate EMI using the formula:

  EMI = ((p * r * ((1 + r)**n))/((1 + r)**n - 1))

  where,

  P: Loan amount, r=rate of interest, n= Loan Tenure

- ***Development of Chatbot Interface:*** We have used flask for web application integration.
- ***Validations and Testing***: This module will check for the testability of the system and ensure that very few to none bugs remain in the developed model. Furthermore, validations can be added to sanitize user input for security purposes.

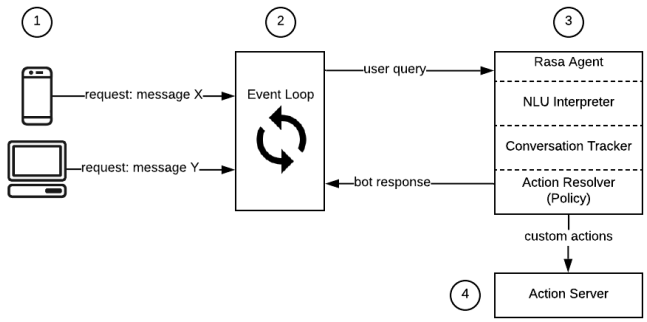
.

IV. TEST CASES

The White Box Testing has been performed in our ML Model and Web application as the internal structure of the application is known to us. Hence, we are easily able to identify the internal error or failures and correct them to ensure a smoother experience for users.

Sr. No	Test Case Name	Purpose of the test	Input	Expected Output
1	Load with the webpage	To test web performance of chatbot	-	Chatbot should get loaded with the webpage
2	Understanding capacity of Chatbot	To test if chatbot identifies words with mistakes like (Imdia or India, Mumbai or Mumbhai)	Mumbhai	Chatbot should sharp enough to understand the word (Mumbai)
3	Input not matching condition	To test conditions where input does not match	Only a number (7)	It should not enter into the loop if required input is not matching or something failed
4	Handling wrong inputs	To test what happens if some text or image something pastes by user	Image	Chatbot should not crash
5	Record chat details	To test if chatbot records chat details	Chat details, database details	Chatbot should forward the chat transcribe to applicable ID's and save it in the database
6	Handling network issues	Need to verify Chatbot behavior in network conditions (no internet)	-	Chatbot shows appropriate error message

MVC Architecture:



V. TESTING THE CHATBOT

The steps taken to test chatbot are illustrated on the following checklist:

- Validate data and stories
- Write test stories
- Evaluate NLU model
- Generating reports with intent classification, entity extraction etc.
- Evaluating dialoe model and checking policy configuration

A. *Confirm data and stories*

This is the first step we have taken before hitting the train button in Rasa. The command

rasa data validate

checks for the inconsistencies in stories or nlu mode

B. *Writing Test Stories*

Test stories are written in the form of exemplary conversations to check whether the bot will behave as expected

Once we have a good set of test cases, we have to run

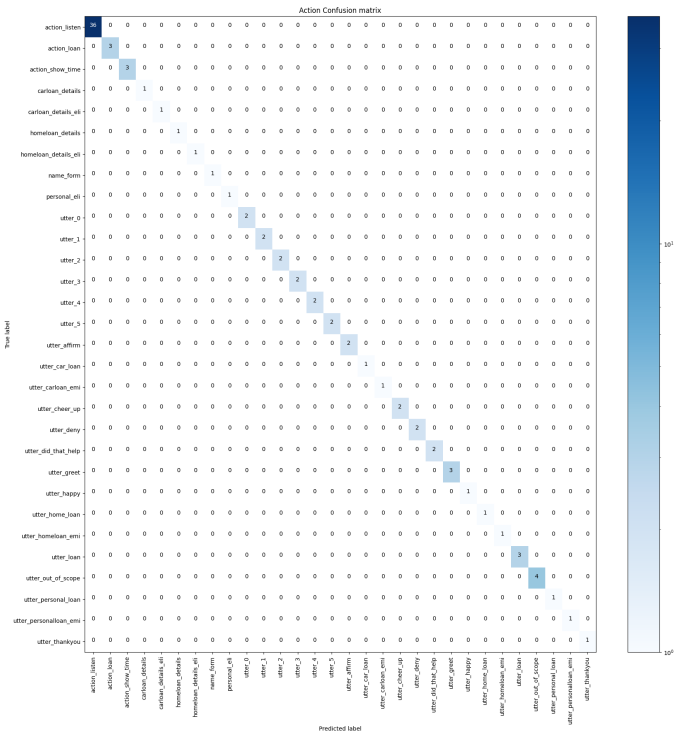
rasa test

```
INFO     rasa.core.test  - Finished collecting predictions.
INFO     rasa.core.test  - Evaluation Results on END-TO-END level:
INFO     rasa.core.test  -  Correct:          4 / 7
INFO     rasa.core.test  -  Accuracy:         0.571
INFO     rasa.core.test  - Stories report saved to results\story_report.json.
INFO     rasa.nlu.test  - Evaluation for entity extractor: TEDPolicy
WARNING  rasa.model_testing  - No labels to evaluate. Skip evaluation.
INFO     rasa.nlu.test  - Classification report saved to results\TEDPolicy_repo
INFO     rasa.nlu.test  - Every entity was predicted correctly by the model.
3\envs\chatbot1\lib\site-packages\rasa\utils\plotting.py:109: UserWarning: Atte
zmin, vmax=zmax),
3\envs\chatbot1\lib\site-packages\rasa\utils\plotting.py:109: UserWarning: Atte
zmin, vmax=zmax),
INFO     rasa.utils.plotting  - Confusion matrix, without normalization:

INFO     rasa.core.test  - Evaluation Results on ACTION level:
INFO     rasa.core.test  -  Correct:          31 / 35
INFO     rasa.core.test  -  F1-Score:         0.941
INFO     rasa.core.test  -  Precision:        1.000
INFO     rasa.core.test  -  Accuracy:         0.902
INFO     rasa.core.test  -  In-data fraction: 0.971
INFO     rasa.utils.plotting  - Confusion matrix, without normalization:
```

[2] *Test Story Results*

story confusion matrix:



C. *Testing NLU Model*

The NLU file contains all the training examples our chatbot is trained with. In the real-life, the bot will encounter examples which are not in the training set of course. That's why we have split your data for testing to simulate that kind of situation.

rasa data split nlu

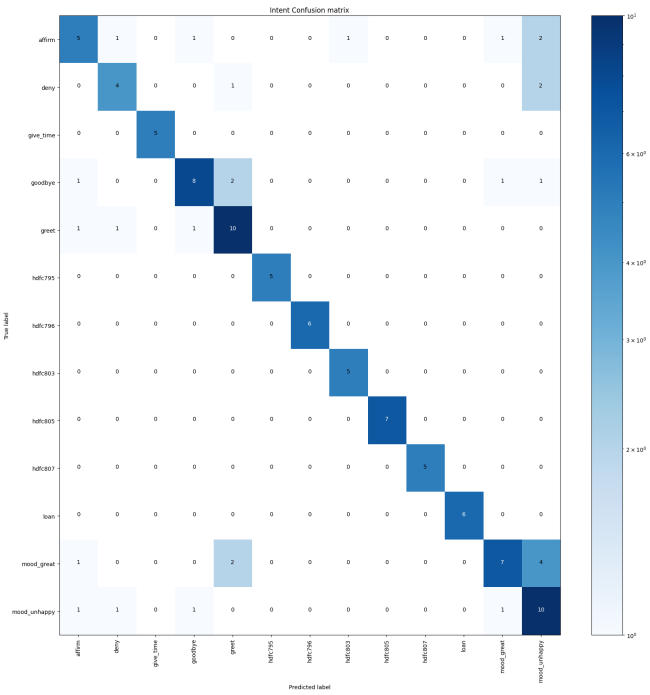
Once our data is split, we can check the prediction rate of our model.

rasa test nlu

```
rasa.model_testing - CV evaluation (n=5),
rasa.model_testing - Intent evaluation results
rasa.nlu.test - train Accuracy: 0.993 (0.006),
rasa.nlu.test - train F1-score: 0.993 (0.006)
rasa.nlu.test - train Precision: 0.994 (0.005)
rasa.nlu.test - test Accuracy: 0.949 (0.069)
rasa.nlu.test - test F1-score: 0.921 (0.058)
rasa.nlu.test - test Precision: 0.928 (0.066)
```

[3] *NLU Result*

Intent Confusion matrix:



D. Evaluating the dialogue model and checking policy configuration

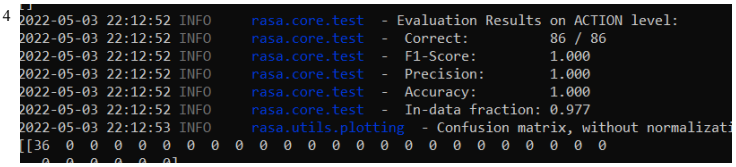
You can check your dialogue model on a group of test stories.

rasa test core --stories test_stories.yml --out results

This command will generate a report about failed stories and a confusion matrix for each story regardless of whether they failed or not.

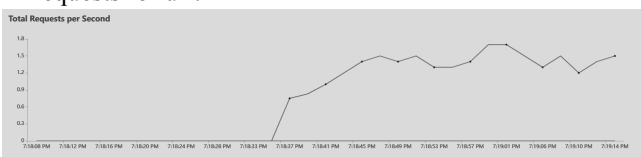
We can test the policy configuration in a similar manner. Policies are pre-trained models you can use in your training. AssesingAssessing their performance will help you decide which configuration is the best for your chatbot.

**rasa train core -c config_1.yml config_2.yml **



E. Load Testing on Chatbot

We have used Locust for load testing, an open source load testing tool. We have defined user behavior with Python code, and swarmed our system with millions of simultaneous users. On testing, we could measure the performance metrics of our chatbot mainly the total requests per second, response time, number of successful and failed requests, exceptions occurred if any etc. We observed that after 91 successful requests, the subsequent requests fail with the following error: 429 Client Error: Too Many Requests for url.



Acknowledgment

We feel obligated to extend our gratitude towards all those who made valuable contributions throughout this duration. We want to express and acknowledge our sincere gratitude towards our guide Dr. Shubhangi Vinayak Tikhe and Prof. Simantinee Kulkarni for their constant guidance and for always giving us constructive advice during the project implementation. Our grateful thanks are due to Dr. M. B. Khambete and Dr. Supriya Kelkar for presenting this opportunity to the students. We are also thankful to the team of Professors for always being there for us and the staff of the department of Computer Engineering for their support and constant cooperation. It is also our duty to thank all our classmates and friends for their help and encouragement during this period. Finally, we express our sincere thanks to all those whose work has helped us in carrying out this project.

References

[1] Nikolaos Polatidis "Chatbot for admissions", University of Birmingham

[2] TheFinancialBrand "Meet 11 of the Most Interesting Chatbots in Banking"

[3] Shashank Bairy R , Rashmi R "A Review of Chatbots in the Banking Sector", IJERTV10IS060203

[4] Hiral Paghadal, Anezka Virani, Apratim Shukla, Dr. G T Thampi "Customer Support Chatbot Leveraging Machine Learning", e-ISSN: 2395-0056

[5] Sahil Bafna, Ashish Kakade, Guided by Dr. Prof. Dhananjay Upasani "Customer Support Chatbot Leveraging Machine Learning", e-ISSN: 2395-0056

[6] Understanding the Basics of Rasa - Open source conversational AI , smazee.com

[4] Policy Configuration Results